

Cost Visibility Dashboard

Build the system that catches a cloud bill going wrong — while there's still time to stop it.

AUTHOR

Kyler Nats

DATE

14 September 2026

STEPS COMPLETED

1 of 24

EVIDENCE CAPTURED

3 of 21 screenshots

Summary

1/24	3	0	0
Steps done	Screenshots	Steps annotated	Done without hints

Notes below are what I wrote while working. Screenshots were taken at the time. Hint counts show where I needed help.

P0

Baseline

Evidence of the 'before'. These can never be retaken once resources exist.

0.1 Capture the empty state **done**

no hints used

TASK

Three screenshots proving nothing was pre-built, plus a machine-readable snapshot of the Azure API.

baseline-empty-rg — Resource groups before the lab

baseline-credit — Owner role and \$200 credit before deployment

baseline-cost-analysis — Cost analysis with no budget and no forecast

Decisions

What I decided, what else I considered, and why. Written down when I made the call, not after.

Build the cost dashboard first, not last

Alternatives: build it last so it has real spend to display.

Chose: first. Set up cost monitoring before spending, not after. It also protects the \$200 credit while I build projects 02-05.

Tradeoff: the dashboard is almost empty at the start.

Forecasted alert alongside actual thresholds

Alternatives: actual-spend thresholds only, which is the default most people configure.

Chose: three actual thresholds (50/80/100%) plus a forecast alert at 100%.

Why: actual thresholds only tell me money is already spent. The forecast alert fires when the month is projected to go over, so I still have time to do something about it.

Cost Management Reader, not Contributor

Alternatives: Reader, or Contributor, on the identity that reads cost data.

Chose: `Cost Management Reader`, scoped to the subscription.

Why: it only needs to read billing data. If it got compromised, all someone gets is a bill. Reader would work too, but it shows every resource's config, which is more access than the job needs.

Storage left with public network access enabled

Alternatives: private endpoint, or service endpoint with a VNet.

Chose: public network access on, with TLS 1.2 minimum, no public blob access, and private container ACLs.

Why: the export service writes from Azure's side. It can't reach a locked down account without private endpoints, and those aren't free.

This is a gap, not a fix. With a budget I'd use a private endpoint.

LRS, not GRS, for the export storage

Alternatives: GRS or ZRS for redundancy.

Chose: LRS.

Why: the export regenerates daily and Azure holds the same data anyway. If I lost a region's copy I'd re-run the export. Paying for geo-redundancy here would be spending money on a problem I don't have.

Redacting identifiers from published evidence

Alternatives: publish screenshots as taken. Microsoft's own documentation shows subscription IDs freely, and a subscription ID is an identifier rather than a credential — it grants nothing on its own.

Chose: blur the subscription ID, account email, and tenant domain

(*.onmicrosoft.com) in every published screenshot, and scrub the ID from captured API output. `snapshot.sh` now does the scrub automatically so it cannot be forgotten later.

Why: there's no reason to publish them. None of the three is secret on its own, but together they're useful for targeted phishing and consent-grant attacks against a tenant. Everything the screenshots are meant to prove still shows.

Method: pixelate first, then blur. A light blur on a fixed-width font can often be read back. Downsampling removes the information instead of hiding it.

Lab log

What I did, in order, including what broke. Written as it happened.

2026-09-09 — Environment setup

Azure CLI install. `brew install azure-cli` — clean, v2.90.0.

`az login --use-device-code` failed.

AADSTS530035: Access has been blocked by security defaults.

Security defaults are on in this tenant and they block device code login.

Microsoft does this on purpose. Device code is a known phishing method, since an attacker can generate a code and get someone to enter it themselves.

Fix: browser login with `az login --tenant <id>`. That flow ties the login to the browser that started it, so it isn't blocked.

I could have turned security defaults off to make the CLI work. Not worth it on the subscription holding my billing data.

Account: Owner on `Azure subscription 1`. I need that level to create the subscription-scope role assignment later.

Entries below are added as the build progresses

2026-09-13 — Step 0: baseline evidence

Three baseline screenshots captured before any resource existed: empty resource group list, subscription showing Owner and \$200 credit, and Cost analysis reading `BUDGET: NONE` / `FORECAST UNAVAILABLE`.

The last one is the useful one. Those two tiles will read `$25` and show a forecast once the budget is deployed, so the before and after are in one image.

Bug: screenshot filenames would not move

Filing them broke in a way that took three wrong guesses to pin down.

```
mv: rename /Users/arfies123/Desktop/Screenshot 2026-09-13 at 1.29.05 PM.png
to 00-baseline-empty-rg.png: No such file or directory
```

`ls` listed the file. `stat`, `cp`, and `head` all said it did not exist. The first theory was macOS TCC protection on `~/Desktop`, since that returns misleading errors for protected paths. Wrong — running outside the sandbox failed identically.

Hexdumping the filename gave it away:

```
$ ls ~/Desktop | grep 2026-09-13 | head -1 | xxd
00000010: 3039 2d31 3320 6174 2031 2e32 392e 3035  09-13 at 1.29.05
00000020: e280 af50 4d2e 706e 670a                ...PM.png
```

`e2 80 af` is U+202F, a narrow no-break space. macOS uses it before AM/PM in screenshot filenames. A typed path with a regular space (`0x20`) does not match, so the file genuinely did not exist under the name being used. Globs matched because the wildcard covered the character, which is exactly why `ls` worked and everything else failed.

Fix: always glob, never type these filenames. `capture.sh` already globbed, so the script was fine; the manual commands were not.

Second bug, same batch: the filing loop indexed a zsh array from 0. zsh arrays are 1-indexed, so the first file was named `.png` and every other name shifted by one. Caught it by checking file sizes against the originals rather than trusting that the move worked.

Takeaway: if a file shows up in `ls` but every other command says it doesn't exist, the name I'm using isn't the name on disk. Check the bytes before assuming it's a permissions problem.